



PLANO DE VOO: DA INTENÇÃO À SPEC EXECUTÁVEL

Uma leitura prática sobre o desenvolvimento orientado a agentes

Heverton Eduardo Peres

ENGENHEIRO DE SOFTWARE & MAKER

Heverton Eduardo Peres

Plano de Voo: Da Intenção à Spec Executável

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

São Paulo
2026

Dados Internacionais de Catalogação na Publicação (CIP)

P573p PERES, Heverton Eduardo

Plano de Voo: Da Intenção à Spec Executável / Heverton Eduardo Peres. –
São Paulo : Fábrica Agêntica de Livros,
2026.

31 p. ; 21 cm.

ISBN 978-65-00000-86-3

1. Plano. 2. Intenção. 3. Spec. 4. Executável. I. Título.

CDD 001.2

Sumário

Plano de Voo: Da Intenção à Spec Executável	6
Capítulo 3: Plano de Voo: Da Intenção à Spec Executável	7
Introdução	7
Explica	7
Ilustra	8
Técnica	9
O Esqueleto da Spec Executável	9
A Regra de Ouro em Ação: Critério que Vira Teste	10
De Spec a Tickets com Bloqueios Explícitos	11
O Formato Canônico da Spec com Casos de Borda	12
A Spec como Fonte de Verdade do Orçamento	12
O Modelo de Rastreabilidade Spec-Teste	13
O Modelo de Ciclo de Vida da Spec	13
O Modelo de Priorização de Requisitos	14
O Modelo de Aceite com Casos de Borda Mínimos	15
O Modelo de Priorização de Requisitos	15
O Modelo de Conflito de Requisitos	16
O Template Universal de Spec	17
O Validador de Spec com Métricas de Qualidade	18
O Caso de Borda como Contrato de Não-Regressão	18
O Caso de Borda como Contrato de Não-Regressão	19
O Modelo de Custo da Spec ao Longo do Ciclo	20
A Regressão de Aceite como Contrato Vivo	21
O Modelo de Priorização de Requisitos por Valor	21
O Vocabulário do Contrato	22
A Regressão de Aceite como Espelho da Spec	22
O Checklist de Qualidade da Spec	22
Passos para Escrever uma Spec Executável	22
Aplica	23
Conclusão	23
Por que este capítulo importa	24
Conceitos-chave deste capítulo	24
Checklist para aplicar hoje	24
Perguntas que você deve se fazer	25

Glossário rápido	25
O erro mais comum nesta fase	25
Um exemplo concreto para fixar	26
A rotina de quem já opera assim	26
O que não fazer: os anti-padrões mais comuns	26
Como medir o progresso na prática	27
O papel do líder neste capítulo	27
Perguntas frequentes honestas	27
Um convite para a prática deliberada	28
Síntese para levar com você	28
De onde veio a necessidade desta mudança	28
Conversando com quem resiste	29
O dia a dia no detalhe	29
O custo invisível que decide tudo	30
Próximos Passos	30

Plano de Voo: Da Intenção à Spec Executável

Uma leitura direta e prática para quem quer levar o desenvolvimento orientado a agentes a sério — sem jargão acadêmico, com exemplos aplicáveis.

Capítulo 3: Plano de Voo: Da Intenção à Spec Executável

Introdução

No Capítulo 2, você ocupou o posto de controlador de voo e aprendeu a separar papéis: o agente executa, a verificação refuta, o humano arbitra. Agora vem o primeiro artefato concreto desse novo ciclo: o plano de voo. Nenhuma aeronave decola sem um plano aprovado — e nenhuma feature agêntica deveria começar sem uma spec executável.

Este capítulo ensina a transformar uma intenção vaga — “quero um sistema de autenticação”, “melhora o carrinho de compras” — em uma spec que vira teste de aceite. Você vai aprender as partes obrigatórias da spec (escopo, requisitos R1..Rn, casos de borda, critérios de aceite), a técnica de decomposição em tickets com bloqueios explícitos, e a regra de ouro: se não dá para escrever o teste de aceite na spec, a spec está incompleta.

Explica

A especificação de software tem uma má reputação justificada. No SDLC clássico, a spec é um documento gigante escrito por analistas, aprovado em reuniões, e abandonado assim que o desenvolvimento começa. O código passa a ser a única fonte de verdade, e a spec vira uma ficção com data de publicação.

O paradigma AI-first recupera a spec — mas não a spec-documento. A spec executável é um **contrato de comportamento** escrito em linguagem que uma máquina consegue validar: requisitos numerados, casos de borda explícitos e critérios de aceite que são, na prática, testes. A diferença é que a spec deixa de ser descrição do que será feito e passa a ser **definição do que conta como pronto**.

Por que isso importa ainda mais com agentes? Porque o agente não tem o contexto tácito que um colega humano de equipe tem. Quando você pede a um desenvolvedor humano “cria a tela de login”, ele preenche mentalmente dezenas de decisões implícitas — onde fica o botão, o que acontece com sessão expirada, como tratar erro de rede. O agente não preenche: ele

inventa, com confiança, as decisões que você não tomou. A spec executável existe para eliminar a invenção.

A pesquisa sobre agentes de engenharia de software confirma o risco. Avaliações como o SWE-bench mostram que agentes resolvem issues com qualidade variável — e que a qualidade despenca quando o problema está mal definido. O problema mal definido não é um bug do agente; é uma lacuna de contrato. A spec fechada reduz dramaticamente a variação de resultado.

Há também uma razão econômica. Tokens são o recurso escasso do ciclo AI-first. Uma spec ambígua gera retrabalho — e cada rodada de retrabalho é uma rodada inteira de tokens consumidos para produzir o que a primeira rodada deveria ter produzido. Escrever a spec bem feita antes do build é a forma mais barata de economizar contexto: custa uma fração do que custaria o ciclo de tentativa e erro do agente.

A técnica de decomposição em tickets com bloqueios explícitos completa o quadro. Uma spec não é só texto: é um grafo de trabalho onde cada tarefa declara o que bloqueia e o que ela bloqueia. “Escrever o teste de aceite do login” bloqueia “implementar o login”; “implementar o login” bloqueia “integrar com o provedor de identidade”. Esse grafo de dependências é o que permite despachar agentes em paralelo com segurança — ninguém começa uma tarefa cujo antecedente não está pronto.

O resultado é uma mudança de mentalidade: a spec não é o início do ciclo, é o **contrato do ciclo**. A fase 2 do SDLC AI-first não termina quando o documento está escrito; termina quando os testes de aceite estão definidos e o grafo de tickets está explícito.

Ilustra

Um plano de voo comercial contém, obrigatoriamente: origem, destino, rota, altitude, velocidade, combustível e alternates — aeroportos para onde o avião pode desviar se algo der errado. O piloto não improvisa o destino durante o voo; o plano é aprovado antes da decolagem e qualquer desvio é negociado com a torre em tempo real.

A spec executável é o plano de voo da feature. O escopo é origem e destino. Os requisitos R1..Rn são a rota e a altitude. Os casos de borda são os alternates — os cenários para onde a implementação desvia quando o caminho feliz falha. E os critérios de aceite são o combustível: a prova objetiva de que o voo pode ser concluído.

[Diagrama do capítulo omitido neste formato.]

Como Comandante de Operações de Software, você vê no diagrama o laço de retorno: a spec só sai do radar quando cada requisito tem um teste de aceite. Caso contrário, volta para a origem — sem vergonha, sem burocracia. É o plano de voo voltando à torre para revisão.

Técnica

O Esqueleto da Spec Executável

A spec executável é um arquivo estruturado que a esteira consegue interpretar. O formato abaixo em YAML cobre as partes obrigatórias: escopo, requisitos, casos de borda e critérios de aceite.

```
espec:
  titulo: "Autenticacao por email e senha"
  versao: "1.0"
  escopo:
    inclui:
      - "Login com email e senha"
      - "Recuperacao de senha"
    exclui:
      - "Login social (OAuth2)"
      - "Autenticacao por biometria"
  requisitos:
    R1: "Usuario cadastrado consegue autenticar com email e senha validos"
    R2: "Senha incorreta gera erro generico, sem revelar se o email existe"
    R3: "Sessao expira apos 30 minutos de inatividade"
  casos_borda:
    - "Email em caixa alta deve ser normalizado para minusculas"
    - "Senha com 5 tentativas falhas bloqueia a conta por 15 minutos"
    - "Sessao expirada durante requisicao retorna 401 e redireciona para login"
  criterios_aceite:
    - "teste: login_fluxo_feliz -> passa quando R1 verificado"
    - "teste: login_senha_errada -> passa quando R2 verificado"
    - "teste: sessao_expirada -> passa quando R3 verificado"
```

A validação da regra de ouro é trivial: se qualquer requisito não tem critério de aceite, a spec está incompleta. O código abaixo faz essa verificação e bloqueia o avanço.

```
from dataclasses import dataclass
from typing import Dict, List

@dataclass
class Requisito:
    id: str
    descricao: str
    criterio_aceite: str = ""
```

```
def validar_spec(requisitos: Dict[str, Requisito]) -> tuple:
    sem_criterio = [
        r.id for r in requisitos.values() if not r.criterio_aceite.strip()
    ]
    if sem_criterio:
        return False, f"requisitos sem teste de aceite: {'',
'.join(sem_criterio)}"
    return True, "spec executavel: todos os requisitos tem criterio de
    aceite"

REQUISITOS = {
    "R1": Requisito("R1", "Usuario cadastrado autentica com credenciais
    validas",
        "login_fluxo_feliz"),
    "R2": Requisito("R2", "Senha incorreta gera erro generico",
    "login_senha_errada"),
    "R3": Requisito("R3", "Sessao expira apos 30 minutos",
    "sessao_expirada"),
}

if __name__ == "__main__":
    ok, motivo = validar_spec(REQUISITOS)
    print(f"[{'OK' if ok else 'BLOQUEADO'}] {motivo}")
```

A Regra de Ouro em Ação: Critério que Vira Teste

O critério de aceite não é uma frase de efeito — é o rascunho do teste. A tradução direta produz o esqueleto do teste que o agente deve escrever (e que deve falhar antes da implementação, no espírito test-first).

```
import unittest

class TesteAutenticacao(unittest.TestCase):
    def setUp(self) -> None:
        self.repositorio = RepositorioUsuariosEmMemoria()

    def test_login_fluxo_feliz(self) -> None:
        self.repositorio.criar("ana@exemplo.com", "segredo-123")
        resultado = autenticar("ana@exemplo.com", "segredo-123")
        self.assertTrue(resultado.autenticado)
```

```

def test_login_senha_errada(self) -> None:
    self.repositorio.criar("ana@exemplo.com", "segredo-123")
    with self.assertRaises(ErroCredenciaisInvalidas):
        autenticar("ana@exemplo.com", "senha-errada")

def test_sessao_expirada(self) -> None:
    sessao = criar_sessao(usuario_id="u1")
    sessao.ultima_atividade = agora() - 31 * 60
    self.assertTrue(sessao.expirada())

if __name__ == "__main__":
    unittest.main()

```

Este código não compila isoladamente (depende de `RepositorioUsuariosEmMemoria`, `autenticar` e `criar_sessao`), mas demonstra o ponto: cada critério de aceite da spec vira um método de teste nomeado pelo mesmo identificador. O agente de build sabe exatamente o que implementar: fazer esses três testes passarem.

De Spec a Tickets com Bloqueios Explícitos

A decomposição em tickets com dependências declaradas permite despacho paralelo seguro. O grafo abaixo em JSON declara o que bloqueia o quê.

```

{
  "tickets": [
    {
      "id": "T1", "tarefa": "escrever teste login_fluxo_feliz",
      "bloqueado_por": [], "bloqueia": ["T4"]},
    {
      "id": "T2", "tarefa": "escrever teste login_senha_errada",
      "bloqueado_por": [], "bloqueia": ["T4"]},
    {
      "id": "T3", "tarefa": "escrever teste sessao_expirada",
      "bloqueado_por": [], "bloqueia": ["T5"]},
    {
      "id": "T4", "tarefa": "implementar autenticar()", "bloqueado_por":
      ["T1", "T2"], "bloqueia": ["T6"]},
    {
      "id": "T5", "tarefa": "implementar sessao e expiracao",
      "bloqueado_por": ["T3"], "bloqueia": ["T6"]},
    {
      "id": "T6", "tarefa": "integrar e rodar suíte completa",
      "bloqueado_por": ["T4", "T5"], "bloqueia": []}
  ]
}

```

O despacho por bloqueios é direto: agentes paralelos pegam T1, T2 e T3 simultaneamente (nenhum bloqueado), e só depois que todos concluem é seguro liberar T4 e T5. Esse controle

de dependência é o que evita que dois agentes editem o mesmo arquivo ao mesmo tempo e corrompam o working tree.

O Formato Canônico da Spec com Casos de Borda

A spec executável se torna robusta quando os casos de borda são escritos no mesmo nível dos requisitos — com identificador, condição, comportamento esperado e o teste que o protege. O formato abaixo é o padrão que a esteira consome:

```
casos_borda:
- id: B1
  condicao: "email em caixa alta"
  comportamento_esperado: "normalizado para minusculas antes da busca"
  teste: "login_email_caixa_alta"
- id: B2
  condicao: "5 tentativas falhas consecutivas"
  comportamento_esperado: "conta bloqueada por 15 minutos"
  teste: "login_bloqueio_apos_5_tentativas"
- id: B3
  condicao: "sessao expirada durante requisicao"
  comportamento_esperado: "resposta 401 e redirecionamento para login"
  teste: "sessao_expirada_durante_requisicao"
- id: B4
  condicao: "usuario com credenciais validas mas conta desativada"
  comportamento_esperado: "resposta generica de credenciais invalidas"
  teste: "login_conta_desativada"
```

Cada caso de borda responde a três perguntas: em que condição, o que deve acontecer e qual teste prova. Quando o agente de build recebe essa spec, ele não precisa adivinhar os cenários — eles estão numerados, esperando virar métodos de teste.

A Spec como Fonte de Verdade do Orçamento

A spec executável também declara o custo de contexto esperado — o combustível que a fase de build consumirá. Quando a spec chega ao build com um orçamento explícito, o agente sabe o teto e o time sabe onde o dinheiro foi parar:

```
{
  "spec": "autenticacao-email-senha",
  "orcamento_build": {
    "tokens_entrada_max": 50000,
    "tokens_saida_max": 15000,
    "estimativa_rodadas": 3
  },
}
```

```

"estimativa_derivada_de": {
  "complexidade": "media",
  "arquivos_envolvidos": 6,
  "testes_planejados": 5
}
}

```

A spec vira o documento único que amarra contrato, critérios e custo — o plano de voo completo, não apenas a rota.

O Modelo de Rastreabilidade Spec-Teste

A rastreabilidade entre spec e teste é a espinha dorsal da Fase 2 — e pode ser verificada por máquina. O modelo abaixo liga requisitos, casos de borda e testes:

```

def verificar_rastreabilidade(requisitos: dict, testes: dict) -> list:
    sem_teste = []
    for rid, crit in requisitos.items():
        if crit not in testes:
            sem_teste.append(rid)
    return sem_teste

REQUISITOS = {"R1": "test_login_fluxo_feliz", "R2":
"test_login_senha_errada",
               "R3": "test_sessao_expirada"}
TESTES = {"test_login_fluxo_feliz": True, "test_login_senha_errada": True}

faltantes = verificar_rastreabilidade(REQUISITOS, TESTES)
if faltantes:
    print(f"Requisitos sem teste: {faltantes}")
else:
    print("Rastreabilidade spec-teste completa")

```

A rastreabilidade verificável fecha o ciclo do capítulo: a spec não termina quando o documento está escrito — termina quando todo requisito tem teste correspondente, e a máquina prova.

O Modelo de Ciclo de Vida da Spec

A spec executável também tem ciclo de vida — nasce, é aprovada, é implementada, é revisada e eventualmente aposenta. O modelo abaixo declara os estados da spec e as transições:

Estado	Significado	Transição para
Rascunho	Em elaboração	Proposta (após critérios completos)
Proposta	Pronta para revisão	Aprovada (após validação)
Aprovada	Contrato vigente	Em implementação (após build iniciar)
Em implementação	Sendo executada	Aprovada (após mudança) ou Aposentada
Aposentada	Fora de escopo	—

O ciclo de vida da spec é o mesmo rigor do ciclo de vida do software: a spec não é um documento congelado, é um contrato vivo com estados e transições auditáveis. A esteira registra cada transição — quem mudou, quando e por quê.

O Modelo de Priorização de Requisitos

Nem todo requisito tem o mesmo peso — e a spec executável declara a prioridade. O modelo MoSCoW adaptado ao AI-first classifica requisitos e define o comportamento da esteira em cada classe:

Classe	Significado	Comportamento da esteira
Must	Sem isso, a feature não existe	Bloqueia build se ausente
Should	Valor alto, contornável	Não bloqueia, mas é critério de release
Could	Valor adicional	Implementado se orçamento permitir
Won't	Fora de escopo agora	Explicitamente declarado como excluído

A classe Won't é a mais negligenciada — e a mais valiosa: declarar o que não será feito elimina a invenção do agente. O modelo abaixo mostra a priorização em formato de máquina:

```
{
  "priorizacao": {
    "MUST": ["R1", "R2", "R3"],
    "SHOULD": ["R4", "R5"],
    "COULD": ["R6"],
    "WONT": ["login social", "biometria", "magic link"]
  },
  "regra": "bloco WONT alimenta a secao Exclui do escopo"
}
```

A priorização é o mapa de combustível da spec: a esteira sabe onde gastar primeiro e onde não gastar nunca.

O Modelo de Aceite com Casos de Borda Mínimos

Um requisito só é aceito quando seus casos de borda mínimos passam. O modelo abaixo associa cada requisito ao seu conjunto mínimo de casos e bloqueia o aceite quando falta algum:

```
CASOS_MINIMOS = {
    'LOGIN-01': ['sucesso', 'senha_incorreta', 'usuario_inexistente',
'conta_bloqueada', 'campo_vazio'],
    'LOGIN-02': ['token_valido', 'token_expirado', 'token_revogado'],
    'LOGIN-03': ['tentativas_abaixo_limite', 'tentativa_no_limite',
'limite_excedido'],
}

def verificar_aceite(requisito, casos_rodados):
    minimos = set(CASOS_MINIMOS.get(requisito, []))
    executados = set(casos_rodados)
    faltantes = minimos - executados
    return {'requisito': requisito, 'aprovado': not faltantes, 'faltantes':
sorted(faltantes)}

print(verificar_aceite('LOGIN-01', ['sucesso', 'senha_incorreta',
'usuario_inexistente', 'campo_vazio']))
```

O conjunto mínimo é o contrato de qualidade da spec: “login funciona” nunca é aceite — “login passou nos cinco casos de borda mínimos” é. Quando o caso de conta bloqueada falta, a resposta não é debate sobre qualidade, é execução do caso. Os conjuntos mínimos são definidos na própria spec (seção de casos de borda) e herdados pelos tickets — a spec não descreve o teste, ela define o critério que o teste verifica.

O Modelo de Priorização de Requisitos

Vamos acompanhar uma transformação concreta: a spec do login. A versão vaga — o que a maioria das organizações escreve — é uma frase: “criar tela de login com email e senha”. Esse é o ticket que gera invenção: o agente decide onde fica o botão, o que acontece com sessão expirada, como tratar erro de rede.

A versão executável é o contrato completo:

```
espec_executavel_login:
  escopo:
    inclui: [login_email/senha, recuperacao_de_senha]
    exclui: [login_social, biometria]
  requisitos:
    R1: "usuario cadastrado autentica com credenciais validas"
    R2: "senha incorreta gera erro generico, sem revelar existencia do
```

```
email"
  R3: "sessao expira apos 30 minutos de inatividade"
casos_borda:
  B1: "email em caixa alta normalizado para minusculas"
  B2: "5 tentativas falhas bloqueiam a conta por 15 minutos"
  B3: "sessao expirada durante requisicao retorna 401"
criterios_aceite:
  - "teste login_fluxo_feliz"
  - "teste login_senha_errada"
  - "teste sessao_expirada"
  - "teste b1_email_caixa_alta"
  - "teste b2_bloqueio_5_tentativas"
```

A diferença entre as duas versões é a diferença entre adivinhar e contratar: o agente recebeu as decisões, não o direito de inventá-las. É essa a transformação que o capítulo ensina — e que se repete em toda feature.

O Modelo de Conflito de Requisitos

Requisitos entram em conflito — e o conflito precisa ser detectado antes da implementação, não durante. O modelo abaixo varre a spec em busca de contradições lógicas e ambiguidades de vocabulário:

```
import re

def detectar_conflitos(requisitos):
    conflitos = []
    for i, r1 in enumerate(requisitos):
        for r2 in requisitos[i + 1:]:
            if 'bloqueia' in r1 and 'permite' in r2 and
extrair_entidade(r1) == extrair_entidade(r2):
                conflitos.append({'tipo': 'contradicao', 'req1': r1[:50],
'req2': r2[:50]})
    return conflitos

def extrair_entidade(texto):
    m = re.search(r'([A-Z-]+)', texto)
    return m.group(1) if m else ''

requisitos = ['PAGAR-01 bloqueia pagamento sem saldo', 'PAGAR-02 permite
pagamento sem saldo em teste']
print(detectar_conflitos(requisitos))
```

O detector é grosseiro — captura só contradições explícitas de vocabulário — mas é o início da disciplina. A maior fonte de conflito em specs reais não é lógica formal, é vocabulário diver-

gente: o mesmo conceito com dois nomes, ou o mesmo nome para dois conceitos. A solução estrutural é o vocabulário ubíquo da parte de arquitetura: antes de escrever requisito, definir o termo no glossário. Conflito detectado na spec custa minutos; conflito detectado em produção custa incidente.

O Template Universal de Spec

Uma organização que produz muitas specs precisa de um template universal — o esqueleto que todo analista e todo agente seguem, reduzindo a variação entre specs. O template abaixo é o padrão:

```
# Spec: <título da feature>

## Intenção (1 parágrafo)
<o que se quer e por quê, em uma frase cada>

## Escopo
- **Inclui:** <lista>
- **Exclui:** <lista – tão importante quanto o inclui>

## Requisitos
| ID | Requisito (testável) | Critério de aceite (nome do teste) |
|----|-----|-----|
| R1 | <frase testável> | <test_aceite_r1> |

## Casos de borda
| ID | Condição | Comportamento esperado | Teste |
|----|-----|-----|-----|
| B1 | <condição> | <comportamento> | <test_b1> |

## Orçamento de contexto
- Tokens de entrada estimados: <N>
- Tokens de saída estimados: <N>
- Rodadas de build estimadas: <N>

## Tickets (grafo de dependências)
| ID | Tarefa | Bloqueado por | Bloqueia |
|----|-----|-----|-----|
| T1 | <tarefa> | – | <T4> |
```

O template é o formulário de plano de voo da organização: padronizado o bastante para ser comparável, flexível o bastante para qualquer feature. A spec que segue o template nasce completa; a que improvisa nasce com lacunas.

O Validador de Spec com Métricas de Qualidade

Uma spec executável pode ser medida. O validador abaixo calcula métricas de qualidade — rastreabilidade, cobertura de casos de borda e ausência de linguagem vaga — e devolve um parecer estruturado que o revisor usa como evidência:

```
import re

PALAVRAS_VAGAS = ['rapido', 'melhor', 'adequado', 'apropriado',
                  'suficiente']

def validar_spec(requisitos, casos_borda, rastreabilidade):
    metricas = {
        'num_requisitos': len(requisitos),
        'num_casos_borda': len(casos_borda),
        'cobertura_casos': len(casos_borda) / len(requisitos) if requisitos
    else 0,
        'rastreabilidade': len(rastreabilidade) / len(requisitos) if
requisitos else 0,
    }
    texto = ' '.join(requisitos).lower()
    vagas = [p for p in PALAVRAS_VAGAS if p in texto]
    metricas['linguagem_vaga'] = vagas
    metricas['aprovada'] = (
        metricas['cobertura_casos'] >= 0.5
        and metricas['rastreabilidade'] >= 1.0
        and not vagas
    )
    return metricas

requisitos = ['CRIAR-CONTA deve validar email', 'CRIAR-CONTA deve exigir
senha forte']
print(validar_spec(requisitos, ['email invalido', 'senha curta'], {'CRIAR-
CONTA': 'R1, R2'}))
```

A métrica de linguagem vaga é a mais reveladora: palavras como “rápido” e “melhor” não definem nada e são impossíveis de testar. Quando o validador detecta uma delas, o requisito volta para o redator com a marcação exata da palavra — a correção é mecânica e não depende de debate.

O Caso de Borda como Contrato de Não-Regressão

A spec executável pode — e deve — ser validada por máquina antes do build. O validador abaixo confere as regras de ouro: requisitos numerados, critérios de aceite e escopo com excluídos:

```

import re

def validar_spec_automatica(texto: str) -> list:
    problemas = []
    requisitos = re.findall(r"\|s*(R\d+)\|s*\|.??\|s*(\S+)\|s*\|", texto)
    bordas = re.findall(r"\|s*(B\d+)\|s*\|.??\|s*(\S+)\|s*\|", texto)
    for rid, criterio in requisitos:
        if not criterio:
            problemas.append(f"{rid} sem criterio de aceite")
    for bid, teste in bordas:
        if not teste:
            problemas.append(f"{bid} sem teste")
    if "Exclui" not in texto:
        problemas.append("escopo sem secao Exclui")
    return problemas

SPEC_EXEMPLO = """
| ID | Requisito | Criterio |
| R1 | login valido | test_login_feliz |
| R2 | sessao expira | test_sessao_expirada |
"""

for problema in validar_spec_automatica(SPEC_EXEMPLO):
    print(f"[ESPEC] {problema}")

```

O validador automático é o primeiro radar da spec: antes de o agente tocar no build, a máquina já conferiu as regras de ouro — e reprovou o que faltar.

O Caso de Borda como Contrato de Não-Regressão

Cada caso de borda aprovado é um contrato de não-regressão: uma promessa de que o comportamento não voltará a quebrar. O padrão técnico é o teste de regressão nomeado com o identificador do caso:

```

import unittest

class TesteCasosBordaAutenticacao(unittest.TestCase):
    def test_b1_email_caixa_alta(self) -> None:
        email = normalizar_email("ANA@exemplo.com")
        self.assertEqual(email, "ana@exemplo.com")

    def test_b2_bloqueio_apos_5_tentativas(self) -> None:

```

```

for _ in range(5):
    tentar_login("ana@exemplo.com", "errada")
with self.assertRaises(ContaBloqueadaTemporariamente):
    tentar_login("ana@exemplo.com", "correta")

def test_b4_conta_desativada(self) -> None:
    with self.assertRaises(ErroCredenciaisInvalidas):
        autenticar("desativada@exemplo.com", "qualquer-senha")

if __name__ == "__main__":
    unittest.main()

```

A ligação é direta: caso de borda B1 da spec → teste `test_b1_email_caixa_alta`. O mapa de rastreabilidade é um-para-um — a auditoria (Fase 2.5) pode conferir que todo caso de borda da spec virou teste.

O Modelo de Custo da Spec ao Longo do Ciclo

A spec não é paga uma vez — ela tem custo de manutenção em todas as fases seguintes. O modelo abaixo estima o custo total de propriedade da spec, contabilizando escrita, revisão, atualização e o custo de um defeito não capturado:

```

def custo_total_spec(caracteres, revisoes_por_mes, meses, taxa_defeito,
                    custo_defeito):
    custo_escrita = caracteres / 500 # 500 caracteres por unidade de
    esforco
    custo_revisao = revisoes_por_mes * meses * 2
    custo_atualizacao = 0.3 * custo_escrita * meses
    custo_defeitos = taxa_defeito * custo_defeito
    total = custo_escrita + custo_revisao + custo_atualizacao +
    custo_defeitos
    return {'escrita': round(custo_escrita, 1), 'revisao': custo_revisao,
            'atualizacao': round(custo_atualizacao, 1), 'defeitos':
    round(custo_defeitos, 1),
            'total': round(total, 1)}

print(custo_total_spec(caracteres=20000, revisoes_por_mes=2, meses=12,
                      taxa_defeito=3, custo_defeito=8))

```

A conta revela a assimetria que justifica o investimento: uma spec de 20 mil caracteres custa pouco para escrever, mas se os 3 defeitos que ela deixou escapar chegarem à produção, cada um custa de 8 a 100 vezes mais para corrigir. O modelo dá ao comandante o número que a conversa de orçamento precisa: spec boa é a que reduz taxa_defeito, e isso se mede no backlog de incidentes.

A Regressão de Aceite como Contrato Vivo

A suíte de aceite não é estática — cresce a cada spec aprovada e a cada incidente (como você verá no Capítulo 8). O padrão técnico é uma suíte única que todos os agentes rodam antes de qualquer merge:

```
#!/usr/bin/env bash
# Regressao de aceite: todos os criterios de todas as specs aprovadas
set -euo pipefail

pytest testes/aceite/ --tb=short --quiet

echo "Regressao de aceite verde: $(find testes/aceite -name 'test_*.py' |
wc -l) testes"
```

A regressão é o elo entre a Fase 2 (spec) e a Fase 5 (verificação): cada critério de aceite da spec vira um teste na regressão, e cada teste na regressão protege uma promessa feita ao negócio.

O Modelo de Priorização de Requisitos por Valor

Quando nem tudo cabe no próximo ciclo, a priorização precisa de critério explícito. O modelo abaixo pontua requisitos por valor de negócio, risco e custo, e devolve a ordem de implementação:

```
def priorizar_requisitos(requisitos):
    for r in requisitos:
        r['score'] = r['valor'] * 0.5 + (5 - r['custo']) * 0.2 + r['risco']
    * 0.3
    return sorted(requisitos, key=lambda x: x['score'], reverse=True)

requisitos = [
    {'id': 'R1', 'valor': 5, 'custo': 2, 'risco': 3},
    {'id': 'R2', 'valor': 4, 'custo': 4, 'risco': 2},
    {'id': 'R3', 'valor': 2, 'custo': 1, 'risco': 5},
]
print(priorizar_requisitos(requisitos))
```

A fórmula expressa a política da organização: valor de negócio pesa mais, risco também entra, custo desconta. O requisito R3 de alto risco sobe na fila apesar do baixo valor — porque risco alto não resolvido cedo vira incidente caro depois. A priorização deixa de ser a opinião do gerente de plantão e vira a execução de uma política registrada.

O Vocabulário do Contrato

A spec executável também carrega o vocabulário ubíquo do domínio — o mesmo glossário que você dominará em profundidade no Capítulo 4. Quando a spec usa “cliente” e o código usa “usuário”, o contrato já nasce rachado: o agente implementa uma coisa, o negócio espera outra, e a integração descobre o conflito. Incluir o glossário na spec — com os termos canônicos e os sinônimos proibidos — é a forma mais barata de alinhar linguagem entre humano, agente e verificação.

A Regressão de Aceite como Espelho da Spec

Uma prática que separa equipes maduras das demais é a regressão de aceite: a suíte de critérios de aceite de todas as specs aprovadas, rodando a cada mudança. Quando uma nova feature altera o comportamento de uma antiga, a regressão acusa o conflito antes do deploy. No contexto agêntico, essa regressão é o radar permanente do contrato: cada critério de aceite vira um teste, e cada teste protege uma promessa feita ao negócio.

O Checklist de Qualidade da Spec

O redator termina a spec com um checklist — e o validador roda o mesmo checklist automaticamente. Os itens: cada requisito tem critério de aceite mensurável, cada critério tem caso de borda mínimo, cada termo técnico está no glossário, nenhuma palavra vaga sobreviveu à revisão. O checklist é curto de propósito: poucos itens, todos verificáveis, nenhum dependente de gosto. Quando o mesmo checklist roda no CI e na cabeça do redator, a spec deixa de depender do humor da revisão e passa a depender de evidência.

Passos para Escrever uma Spec Executável

1. **Escreva a intenção em 1 parágrafo** — se não couber em 1 parágrafo, a intenção ainda é duas intenções.
2. **Defina escopo incluído e excluído** — o que está fora é tão importante quanto o que está dentro.
3. **Numere os requisitos R1..Rn** em frases testáveis (“o usuário consegue...”, “o sistema retorna...”).
4. **Liste casos de borda** — comece pelos que você já viu quebrar em produção.
5. **Escreva um critério de aceite por requisito** — no formato de nome de teste.
6. **Decomponha em tickets** com `bloqueado_por` / `bloqueia` explícitos.
7. **Valide a regra de ouro** com o script acima antes de autorizar o build.

Aplica

Cena real, em segunda pessoa. Você é o product manager técnico de um SaaS de RH. O CEO pede, com urgência, uma “integração com o novo provedor de folha de pagamento” — sem mais detalhes. No SDLC clássico, você abriria um épico no Jira, e o time começaria a “investigar” a integração, consumindo dias de trabalho exploratório.

No SDLC AI-first, você aplica o que aprendeu. Primeiro, transforma a urgência em escopo: “integrar o envio de faturas de folha para o provedor X, com retry e idempotência”. Depois, em uma sessão de 40 minutos, escreve a spec executável com requisitos (R1: enviar fatura; R2: retry com backoff; R3: idempotência por id de fatura), casos de borda (provedor fora do ar, fatura duplicada, timeout parcial) e critérios de aceite nomeados.

O erro comum — e você quase caiu nele — é pular direto para “pedir ao agente para implementar”. Você sabe que o resultado seria um agente inventando decisões de contrato: o que acontece com fatura duplicada? Qual timeout? Quantas tentativas? A spec força essas respostas antes do primeiro token de implementação.

O diagnóstico: a intenção vaga era o problema, não a integração. A correção: a spec fechou o contrato e o agente executou a implementação em uma fração do tempo — porque não precisou adivinhar nada.

Na prática, seu checklist para toda nova feature: intenção em 1 parágrafo, escopo com excluídos, requisitos numerados, casos de borda, critérios de aceite com nome de teste, grafo de tickets. Se qualquer item falta, a feature não decola.

Armadilhas comuns: escrever specs longas demais (a spec executável é curta — páginas, não capítulos); confundir descrição com critério (“o sistema deve ser seguro” não é critério; “sessão expira em 30 minutos” é); e permitir que o agente “complete” a spec durante o build — o completar é sua prerrogativa, não dele.

Conclusão

Você escreveu o primeiro plano de voo. Três marcos: primeiro, a spec executável como contrato de comportamento — escopo, requisitos, casos de borda e critérios de aceite — em vez de documento descritivo; segundo, a regra de ouro implementada em código: sem critério de aceite, sem decolagem; terceiro, a decomposição em tickets com bloqueios explícitos, que habilita o despacho paralelo seguro de agentes.

Como desafio, pegue a última feature que sua equipe entregou e reconstrua sua spec executável a partir do que foi feito. Você vai descobrir quais decisões foram tomadas por acidente, e que deveriam ter sido tomadas por contrato.

No próximo capítulo, você vai desenhar a cartografia do domínio: design orientado a agentes, fronteiras de módulos e vocabulário ubíquo — o mapa que os agentes usarão para navegar.

Por que este capítulo importa

Se você chegou até aqui, já percebeu que plano de voo: da intenção à spec executável. Este capítulo — *Capítulo 3: Plano de Voo: Da Intenção à Spec Executável* — é um convite para parar de usar IA como ferramenta de autocomplete e começar a tratá-la como parte estrutural do seu processo de desenvolvimento. A diferença entre as duas posturas é exatamente o que separa quem apenas acelera o que já fazia de quem repensa o que é possível.

Vamos explorar isso com exemplos práticos, código real e um passo a passo que você pode aplicar ainda hoje, sem esperar por infraestrutura nova ou aprovação de comitê. A ideia é simples: cada seção termina com algo que você pode executar em menos de uma hora.

Conceitos-chave deste capítulo

- **O contrato antes da execução:** antes de deixar qualquer agente trabalhar, defina o que significa “feito” em termos verificáveis.
- **Evidência antes de afirmação:** o que não pode ser verificado não pode ser delegado com segurança.
- **Aprendizado contínuo:** cada entrega, boa ou ruim, é matéria-prima para o próximo ciclo.

Esses três conceitos aparecem, de formas diferentes, em todas as seções a seguir. Mantê-los em mente enquanto você lê vai transformar exemplos isolados em um padrão que você reconhece na sua própria rotina.

Checklist para aplicar hoje

1. Escolha uma tarefa pequena e bem definida do seu backlog.
2. Escreva o critério de aceite em uma frase verificável.
3. Delegue a execução a um agente, mantendo a verificação em suas mãos.
4. Registre o que funcionou e o que não funcionou.
5. Transforme o aprendizado em um procedimento reutilizável.

Se você fizer apenas o primeiro item, já estará à frente da maioria das equipes — que continua discutindo IA em reuniões sem nunca definir o que quer que ela faça.

Perguntas que você deve se fazer

1. Qual fase do meu processo consome mais tempo hoje — e por quê?
2. O que eu delegaria a um agente amanhã se tivesse certeza de que o resultado seria verificado?
3. Qual informação eu poderia registrar hoje que tornaria a próxima iteração mais barata?
4. Quem no meu time revisa o trabalho de quem — e com qual critério?
5. O que eu faria se o custo de cada tentativa caísse para quase zero?

Essas perguntas não têm resposta certa, mas têm uma propriedade em comum: elas forçam você a sair da conversa abstrata sobre IA e entrar no terreno do seu processo real. E é exatamente nesse terreno que o SDLC AI-first produz resultado.

Glossário rápido

- **Agente:** programa que usa um modelo de linguagem para planejar e executar tarefas com acesso a ferramentas.
- **Harness:** a camada que conecta o agente ao ambiente — arquivos, comandos, testes e regras.
- **Spec executável:** especificação cujos critérios podem ser verificados por máquina.
- **Verificação adversarial:** camada que refuta o trabalho produzido, em vez de apenas confirmá-lo.
- **Contexto:** a janela de informação que o modelo enxerga a cada passo — o recurso mais caro do ciclo.

Dominar esses cinco termos é suficiente para acompanhar qualquer discussão séria sobre desenvolvimento orientado a agentes.

O erro mais comum nesta fase

A maioria das equipes comete o mesmo erro: adota a ferramenta e mantém o processo. O agente entra no fluxo como um autocomplete sofisticado, e todo o potencial de transformação se perde em pequenas conveniências. O antídoto é simples e desconfortável: mude o processo primeiro, depois traga a ferramenta. Defina o contrato, o critério de aceite e a verificação antes de permitir

que o agente produza em escala. É contra intuitivo, mas é o que separa as equipes que capturam valor das que apenas geram volume.

Um exemplo concreto para fixar

Imagine uma pequena feature de faturamento. No fluxo tradicional, um desenvolvedor recebe a tarefa, interpreta a intenção, escreve o código e um revisor confia na leitura. No fluxo orientado a agentes, a mesma feature começa com uma frase verificável: “o valor total deve considerar o desconto aplicado antes dos impostos”. O agente implementa, os testes verificam a regra, e o humano revisa a evidência — não o código linha a linha, mas o comportamento observado. Perceba o deslocamento: o humano deixa de ler tudo para auditar o essencial, e o agente deixa de adivinhar para executar contra um critério. É essa troca que o restante do livro explora em profundidade.

A rotina de quem já opera assim

Uma semana de trabalho em uma equipe que já adotou o ciclo orientado a agentes não parece uma revolução — parece um fluxo calmo e bem definido. Na segunda-feira, a especificação da semana é revisada em uma reunião curta: cada critério de aceite é lido em voz alta e qualquer ambiguidade é resolvida antes de tocar em código. Na terça, os agentes executam as tarefas em isolamento, enquanto os humanos revisam a arquitetura e os contratos. Na quarta, a verificação roda: testes, revisão adversarial e a decisão de merge apoiada em evidência. Na quinta, o que passou vai para produção em canário, com observabilidade ligada. Na sexta, o debriefing transforma os incidentes da semana em lições e skills. Nenhum dia é heroico; todos os dias são previsíveis. E é exatamente essa previsibilidade — não a velocidade máxima — que define a alta performance.

O que não fazer: os anti-padrões mais comuns

Se você quer destruir o valor do desenvolvimento orientado a agentes, aqui estão as receitas mais eficientes. Primeiro, o prompt-and-pray: gere o código, olhe por cima, e peça desculpas quando quebrar. Funciona em demos, falha em produção. Segundo, a spec decorativa: escreva documentos longos que ninguém verifica e que o agente não consegue executar — o pior dos dois mundos. Terceiro, a auto-verificação: deixe que quem escreveu valide o próprio trabalho, sem revisor independente; é a forma mais rápida de transformar confiança em acidente. Quarto, a

delegação sem observabilidade: conceda autonomia sem instrumentar o comportamento. Todos esses padrões têm uma origem comum — a pressa em capturar o ganho sem construir o controle. E todos têm o mesmo antídoto: contrato antes de execução, evidência antes de afirmação, e revisão independente em toda entrega.

Como medir o progresso na prática

Uma dúvida legítima é: como saber se a adoção está dando certo? Métricas tradicionais de velocidade podem enganar — um time pode entregar mais rápido e acumular dívida técnica invisível. O indicador mais confiável no ciclo orientado a agentes é a estabilidade: quantos incidentes em produção, quanto tempo de retrabalho, quantas correções de emergência. Um segundo indicador é o custo de contexto: quantos tokens cada fase consome, e onde o desperdício se concentra. Um terceiro é a taxa de aceite na primeira verificação: se os agentes precisam de muitas rodadas de refutação, o contrato está fraco — o problema não é o agente, é a spec. Com esses três números na mesa, a conversa de progresso deixa de ser anedótica e vira análise de processo.

O papel do líder neste capítulo

Nada do que este capítulo descreve acontece por acaso — alguém precisa criar as condições para que o processo exista. Esse alguém é o líder técnico, o líder de equipe ou o arquiteto que decidiu tratar o ciclo orientado a agentes como uma mudança de processo, não como a instalação de uma ferramenta. O trabalho do líder aqui tem quatro frentes. A primeira é a modelagem do contrato: garantir que cada fase tem entrada, saída e critério definidos. A segunda é a calibragem da confiança: decidir o que pode ser delegado e o que exige decisão humana, e documentar essa decisão. A terceira é a defesa do tempo de verificação: em uma cultura que celebra velocidade, o líder precisa defender o orçamento de revisão como quem defende o seguro do prédio. A quarta é o exemplo: o líder que pede evidência antes de afirmar, em toda reunião, ensina mais do que qualquer documento de processo.

Perguntas frequentes honestas

P: Isso não vai tirar o emprego dos desenvolvedores? R: A história do ciclo de vida nunca foi sobre menos trabalho humano, mas sobre trabalho mais valioso. O que muda é a natureza da tarefa: escrever código repetitivo deixa de ser o centro, e a especificação, a verificação e o

desenho de contratos ocupam o lugar. P: Precisamos de um time de especialistas em IA para começar? R: Não. Precisa-se de disciplina de processo e de vontade de medir. As ferramentas evoluem rápido; o processo é o que permanece. P: E se o agente produzir código que ninguém entende? R: Essa é a pergunta certa — e a resposta é a verificação: se o código passa nos testes, na revisão adversarial e na observabilidade em produção, o fato de ter sido escrito por um agente é irrelevante. O critério não é a origem, é a evidência. P: Quanto tempo leva para ver resultado? R: Na primeira semana você já vê o efeito de escrever critérios de aceite verificáveis, independentemente de agentes. Os ganhos estruturais aparecem em um a dois ciclos.

Um convite para a prática deliberada

Conhecimento sem prática é entretenimento disfarçado de aprendizado. Este capítulo termina com um convite para a prática deliberada: escolha um artefato real do seu trabalho — uma spec, um teste, um release — e aplique deliberadamente um dos conceitos aqui descritos. Anote o antes e o depois. Repita por quatro semanas. No fim do mês, compare: o processo está mais previsível? O custo de contexto caiu? A estabilidade melhorou? Esse experimento pessoal, pequeno e mensurável, vale mais do que qualquer curso. É assim que o ciclo orientado a agentes deixa de ser um conceito que você explica para outras pessoas e se torna uma capacidade que você demonstra.

Síntese para levar com você

Se você guardar apenas uma ideia deste capítulo, que seja esta: no ciclo orientado a agentes, o contrato precede a execução, a evidência precede a afirmação e a revisão independente precede a entrega. Tudo o mais — as ferramentas, os modelos, os fluxos — muda rápido e pode ser aprendido conforme a necessidade. O que não muda é a disciplina: sem ela, a IA é um gerador de volume; com ela, é um multiplicador de capacidade. O resto do livro é a expansão dessa disciplina em cada fase do ciclo de vida.

De onde veio a necessidade desta mudança

Vale a pena entender por que este capítulo existe — e por que ele não foi escrito dez anos atrás. A resposta está na economia do desenvolvimento de software. Durante décadas, o custo dominante de produzir software foi o trabalho humano: escrever, revisar, corrigir. Todo o ciclo de vida clássico foi desenhado em torno dessa escassez — processos, papéis e artefatos existem

para coordenar pessoas e evitar retrabalho caro. O que mudou nos últimos anos foi a emergência de modelos capazes de gerar, revisar e executar código com custo marginal próximo de zero. De repente, a escassez dominante não é mais a mão de obra: é a capacidade de especificar, orquestrar e verificar. Esse deslocamento — de horas-homem para tokens e contexto — é a raiz de tudo o que este capítulo descreve. Quem entende essa mudança de economia entende por que o processo precisa mudar junto com a ferramenta.

Conversando com quem resiste

Em toda equipe há quem resista à mudança — e a resistência quase nunca é preguiça, é uma pergunta legítima sem resposta. As objeções mais comuns são três. “Já tentamos automação e quebrou”: a resposta é que a automação anterior quebrou porque o processo não tinha contrato nem verificação; é exatamente isso que o novo ciclo constrói antes de automatizar. “IA gera código que ninguém entende”: a resposta é que o critério de entendimento mudou — o que importa não é a origem do código, mas se ele passa na verificação; e a revisão de contrato e arquitetura continua humana. “Isso é modismo”: a resposta mais honesta é que pode ser, mas o processo que este capítulo descreve — especificar, delegar, verificar, aprender — melhora o ciclo com ou sem IA. A disciplina é o investimento à prova de modismo.

O dia a dia no detalhe

Para tornar concreto o que este capítulo descreve, vale percorrer o dia a dia de uma tarefa típica, passo a passo. A manhã começa com a revisão da intenção: o produto explica o que quer, o time traduz em requisitos com critérios verificáveis e ninguém toca em código antes de a spec estar aprovada. Na sequência, o trabalho é despachado: cada tarefa vai para um contexto isolado, com seu contrato anexado. A execução produz artefatos — código, testes, diagramas — e cada artefato carrega a evidência de como foi produzido. A tarde é de verificação: testes automáticos, revisão adversarial e a leitura humana do que é crítico. O que passa, segue; o que não passa, volta com o parecer anexado — sem discussão de opinião, porque o critério já estava escrito. O fim do dia é de registro: o que foi aprendido, o que custou em contexto, o que deve mudar no processo. Esse fluxo parece simples, mas cada passo exige disciplina — e é exatamente a simplicidade do ritmo que o torna sustentável.

O custo invisível que decide tudo

Há um recurso que atravessa todos os exemplos deste capítulo e que raramente aparece nas discussões: o contexto. Cada interação com um modelo de linguagem consome uma janela de informação — e essa janela é limitada e cara. Uma spec mal escrita gasta contexto em ciclos de correção. Um log inteiro no contexto gasta contexto que poderia servir à verificação. Uma busca redundante gasta contexto sem produzir informação. Quem ignora esse custo descobre, cedo ou tarde, que a automação ficou mais cara que o trabalho manual que pretendia substituir. Por isso a disciplina de contexto não é um detalhe de economia — é uma decisão de arquitetura do ciclo. Medir o consumo por fase, comprimir o que é ruído e injetar apenas o necessário são práticas que determinam se o SDLC AI-first se sustenta em escala. Este capítulo toca nesse tema; os capítulos finais do livro o desdobram em técnica.

Próximos Passos

Você acabou de percorrer um dos capítulos centrais do SDLC AI-first. Se este conteúdo fez sentido para você, o próximo passo natural é continuar a jornada pelo livro completo, que aprofunda cada fase do ciclo: especificação executável, design de domínio, harness e agentes, verificação adversarial, entrega segura, aprendizado contínuo, economia de tokens e governança de maturidade.

Enquanto isso, aqui vão três ações concretas:

1. **Pratique em um projeto pequeno.** Nada substitui experimentar com as próprias mãos — escolha um repositório pessoal e aplique um dos conceitos deste capítulo.
2. **Formalize seu contrato.** Escreva o critério de aceite de uma tarefa real antes de delegá-la. É um exercício de cinco minutos que muda completamente a qualidade do resultado.
3. **Mensure seu processo.** Registre quanto tempo e quanto contexto cada fase consome. O que não é medido não pode ser melhorado.

O céu do software agora tem torres de controle. O próximo voo é seu.

Boa leitura e bons voos.

— Heverton Eduardo Peres.

